

chapter13

제13장 피처 거버넌스

8장에서 피처를 “한 번 정의하고, 한 곳에 저장하고, 훈련과 서빙이 같은 곳에서 조회한다”는 구조로 만들었다. 그때는 피처가 하나(전일 민원 건수)이고 소비자도 사실상 하나였다 — 14장의 통합 MVP가 8장의 피처 스토어 없이도 운영되었던 이유가 이것이다(표 14.5의 “피처 1종·소비자 1개”). 그러나 피처가 여러 종류가 되고 여러 모델·부서가 같은 피처를 재사용하기 시작하면, 8장의 기술적 구조만으로는 부족한 질문들이 생긴다: 이 피처는 정확히 무엇이고 누가 책임지는가, 누가 언제 무슨 목적으로 조회했는가, 정의가 바뀌면 어느 모델이 영향을 받는가. 이 질문들에 기록과 통제로 답하는 것이 피처 거버넌스다. 이 장은 거버넌스를 세 장치로 나눈다 — **카탈로그**(무엇인가), **접근 제어와 사용 이력**(누가, 언제, 왜), **품질 점검과 변경 관리**(맞는가·바뀌면 어떻게). 실습(★★★☆☆)은 8장이 registry에서 추출해 남긴 피처 정의와 7장 확정 집계를 입력으로, 이 세 장치를 SQLite와 표준 라이브러리만으로 독립 구현한다 — Feast를 다시 설치하지 않고도 거버넌스의 규율을 실측으로 만든다.

학습 목표

피처 거버넌스의 세 요소(카탈로그·접근 이력·품질 변경)가 각각 어떤 질문에 답하는지 설명할 수 있다.
역할 기반 접근 제어와 감사 로그를 설계하고, 로그에서 감사 점검표를 자동 생성할 수 있다.
피처 품질 규칙(계약)과 분포 감시(드리프트)를 구분하고, 정의 변경의 영향 소비자를 식별할 수 있다.
요구사항에서 접근 등급 체계와 품질 규칙 임계값을 스스로 정하고, 각 값의 근거와 그 설계가 깨지는 조건을 적을 수 있다.
이 장의 거버넌스 장치를 민간 데이터 플랫폼으로 옮길 때 무엇이 그대로이고 무엇이 바뀌는지 판단할 수 있다.

13.1 피처 정의, 소유자, 사용 목적

학습 목표

피처가 재사용 자산이 되는 순간 왜 “정의만으로는 부족”한지 이해한다.
피처의 신원을 이루는 항목(정의·소유자·목적)을 구분한다.

직관: 피처에도 신원이 필요하다

8장의 피처 정의서(표 8.4)는 이미 스키마와 소유자를 담고 있었다. 거버넌스는 그 정의서를 **여러 소비자가 공유하는 계약**으로 격상시키는 일이다. 스키마 정보에 이름(정의)·책임자(소유자)·용도(사용 목적)·발급 이력(버전·변경)이 더해져야 신원이 완성된다. 정보만 있고 신원이 없으면, 그 피처가 틀렸을 때 누구에게 물어야 하는지, 바뀌었을 때 누구에게 알려야 하는지 아무도 모른다.

피처의 신원은 세 축으로 이루어진다.

정의(definition): 어떤 산식·스키마로 계산되는가. 실습 피처라면 `complaint_count` 는 “전일 확정 민원 건수(7장 일별 집계)”, `day_unmapped_rate` 는 “전일 정제 품질 — 매핑 실패율”이다.

소유자(owner): 정의와 품질에 책임지는 부서/담당. 원천이 바뀌면 소비자에게 알릴 의무가 여기에 붙는다.

사용 목적(purpose): 이 피처가 무엇을 위해 존재하는가. 목적은 나중에 접근 제어의 기준이 된다 — 목적 밖 이용을 막는 것이 개인정보 보호법 제18조(목적 외 이용·제공 제한)의 핵심이기 때문이다.

거버넌스 없는 피처의 위험 — 8장이 이미 보여 줬다

8장은 거버넌스의 필요를 실측으로 미리 증언했다. “각자 계산”이 만든 16·17·18의 스큐(표 8.1의 워크드 예제), `materialize` 창을 벗어나 조용히 비었던 온라인 스토어, 시점 조인에서 소리 없이 사라진 한 행 — 세 사고 모두 “이 피처가 무엇이고 누가 보증하는가”가 기록되지 않아서, 또는 기록과 실물이 어긋나서 생겼다. 거버넌스는 이 위험을 문서가 아니라 구조로 통제한다. 정의는 수작업으로 쓰지 않고 registry에서 추출하고(8장 패턴), 접근은 로그로 남기며, 변경은 버전으로 말한다.

표 13.1 피처 거버넌스의 세 요소

요소	답하는 질문	이 장의 절	실습 장치
카탈로그	이 피처는 무엇이고 누가 책임지는가	13.1·13.2	SQLite <code>feature_catalog</code>
접근 제어·사용 이력	누가 언제 무슨 목적으로 조회했는가	13.3	<code>access_log</code> + 정책
품질·변경 관리	맞는가, 바뀌면 어떻게 하는가	13.4	품질 규칙 + <code>change_log</code>

세 장치는 데이터 관리의 오래된 규율이다

피처 거버넌스의 세 장치는 MLOps가 새로 발명한 것이 아니라, 데이터 관리 분야가 오래 다듬어 온 규율을 “피처”라는 대상에 적용한 것이다. 데이터 관리 지식체계(DAMA-DMBOK)는 데이터 거버넌스·메타데이터 관리·데이터 품질을 각각 독립된 지식 영역으로 정의하는데, 이 장의 세 장치가 그 셋에 그대로 대응한다 — 카탈로그는 메타데이터 관리(무엇이 있고 무엇을 뜻하는가), 접근 제어·사용 이력은 데이터 거버넌스(누가 권한을 갖고 누가 책임지는가), 품질 점검·변경 관리는 데이터 품질(값이 계약을 지키는가)이다. 이 대응을 파악하면 실무에서 불필요한 작업을 줄인다 — 피처 스토어를 새로 도입하는 조직이 “피처 전용 거버넌스”를 처음부터 구축하지 않아도 되기 때문이다. 기관에 이미 데이터 거버넌스 체계가 있다면 피처는 그 체계가 관리하는 또 하나의 데이터 자산일 뿐이고, 없다면 이 세 규율이 곧 만들어야 할 최소 골격이다. (DAMA-DMBOK은 피처가 아니라 데이터 일반의 관리 프레임이므로 직접 선례가 아니라 배경 원칙으로 인용한다.)

공공 맥락에서는 여기에 AI 위험관리의 언어가 겹친다. 1장에서 소개한 NIST AI 위험관리 프레임워크(AI RMF)의 네 기능 중 GOVERN(체계)은 “누가 책임지고, 무엇을 기록하며, 어떤 정책으로 통제하는가”를 요구한다. 피처 거버넌스의 소유자 지정·접근 로그·변경 승인이 바로 그 GOVERN을 피처 층에서 구체화한

것이다 — 발주·검수 문서가 “AI를 책임 있게 운영할 것”이라는 추상 요구를 적을 때, 그것이 피처 층에서 검수 가능한 산출물(카탈로그·로그·변경 이력)로 전환되는 지점이 이 장이다.

민간에서 공공으로: 규제가 먼저 만든 거버넌스

피처 거버넌스의 장치 — 카탈로그·계보(lineage)·접근 통제 — 는 민간에서, 특히 금융권에서 규제 대응으로 먼저 성숙했다. 은행은 리스크 데이터 집계 원칙(BCBS 239)에 따라 “이 리스크 지표가 어느 원천에서 어떤 변환을 거쳐 나왔는가”를 계보로 증명해야 하고, 개인정보를 처리하는 기업은 처리활동 기록(EU GDPR 제30조)으로 “어떤 데이터를 무슨 목적으로 처리하는가”를 상시 문서화해야 한다. 데이터 카탈로그와 lineage 도구가 민간에서 먼저 성숙한 데에는 이 규제 압력이 중요한 배경 중 하나로 작용했으며(대규모 데이터의 발견·재사용 수요도 함께였다), 광고·추천의 실시간 피처 서빙에도 같은 계보·접근 통제가 그대로 붙는다. 즉 민간의 실시간 운영은 “빠르게 공급하되 출처와 접근을 증명 가능하게” 하는 두 요구를 동시에 충족해야 한다.

공공으로 옮겨 오면 강조점이 이동한다. 민간의 1차 관심이 “누가 이 값을 써서 얼마의 수익·성능을 냈는가”라면, 공공의 1차 관심은 “누가 언제 왜 이 데이터를 봤는가”다 — 형평성·책임성·감사가 값의 효용보다 앞선다. 그래서 공공 피처 거버넌스는 민간 기법을 세 가지로 변형한다. 첫째, 허용 로그만이 아니라 **거부 로그**까지 감사 증거로 명시 관리한다(민간에서는 성능·수익 분석 로그가 전면에 놓이기 쉽다). 둘째, 접근의 목적을 등록·대조해 목적 외 이용을 차단한다(피처가 개인정보를 담으면 개인정보 보호법 제18조의 목적 외 이용·제공 제한이 직접 적용된다). 셋째, 감사 점검표를 사람의 주장이 아니라 로그에서 자동 생성해 “지켰다”는 진술이 개입할 여지를 제거한다. 민간 데이터 거버넌스를 익힌 실무자가 공공으로 옮겨올 때 새로 배울 것은 도구가 아니라 이 세 변형이고, 반대로 공공을 먼저 배운 학습자가 민간으로 가면 같은 카탈로그·로그·계보가 이미 규제 대응의 언어로 존재함을 발견한다.

공공 사례

복지 사각지대 발굴 모델과 재난 대응 모델이 같은 “가구별 공과금 체납” 피처를 재사용한다고 가정한다. 정의서만 있고 신원이 없으면, 통계 부서가 체납 산정 방식을 바꿨을 때 두 모델이 그것을 모른 채 낡은 가정을 계속 쓴다. 소유자·목적·버전이 붙어 있으면 계약이 성립한다 — 소유 부서는 변경 시 통보하고, 각 모델은 자신의 목적이 그 피처의 사용 목적과 맞는지 판단한다. 검수 관점의 질문: 이 피처의 소유자는 명시되어 있는가, 그리고 우리 모델의 사용이 그 피처의 등록된 목적 안에 있는가. ### 핵심 정리 - 거버넌스는 8장의 피처 정의를 여러 소비자가 공유하는 계약으로 격상시킨다. - 피처의 신원은 정의·소유자·사용 목적으로 이루어지며, 목적은 접근 제어의 기준이 된다. - 거버넌스는 위험을 문서가 아니라 구조(추출·로그·버전)로 누른다.

13.2 피처 카탈로그

학습 목표

데이터 카탈로그가 담는 메타데이터 항목을 안다.

카탈로그를 수동 작성하지 않고 실제 정의·소비 사실에서 생성하는 이유를 이해한다.

핵심 개념: 카탈로그는 피처의 주소록

카탈로그는 “어떤 피처가 있고, 각각 무엇인가”를 한곳에 모은 등록소다. 운영에서는 DataHub나 OpenLineage 같은 도구가 데이터셋·피처·대시보드의 메타데이터와 계보(lineage)를 카탈로그화한다 (보충). 규모와 무관하게 카탈로그가 담는 항목은 대체로 같다.

정의·스키마: 산식과 자료형(dtype).

소유자·사용 목적: 책임과 용도.

원천·갱신 주기: 어느 산출물에서, 얼마나 자주.

개인정보 등급: 피처 단위 민감도(8.4의 등급 설계).

소비자: 이 피처를 읽는 모델·서비스 — 변경 영향 분석의 근거가 된다.

타임스탬프 의미·TTL·버전: 8장에서 확정한 시간 규약과 버전.

현장의 카탈로그 도구는 역할이 다르다

실습은 SQLite 한 테이블로 카탈로그를 만들지만, 운영에서는 목적이 다른 도구들이 층을 이룬다 — 이름이 비슷해 혼동하기 쉬우므로 역할을 구분해 두는 편이 좋다. **DataHub**는 데이터셋·피처·대시보드의 메타데이터를 한곳에 모아 “무엇이 있고 누가 쓰는가”를 검색하게 하는 메타데이터 카탈로그다.

OpenLineage는 도구가 아니라 개방 표준으로, “이 산출물이 어느 작업에서 어떤 입력으로 나왔는가”라는 계보 이벤트를 도구 간에 같은 형식으로 주고받게 한다 — 계보가 특정 벤더에 종속되지 않도록 만든 표준이다. **Amundsen**(Lyft에서 시작)은 데이터 발견(discovery)에 특화되어, 분석가가 “필요한 데이터가 어디 있는지”를 빠르게 찾도록 돕는다. 셋의 공통 지향이 이른바 액티브 메타데이터(active metadata)다 — 메타데이터를 사람이 가끔 들여다보는 정적 문서가 아니라, 파이프라인이 실행될 때마다 자동으로 갱신되고 변경 영향을 하류에 알리는 메타데이터로 다루는 것이다. 실습의 “카탈로그를 수동 작성하지 않고 실제 산출물에서 생성한다”는 원칙이 바로 이 액티브 메타데이터의 축소판이다 — 규모만 다를 뿐, 문서가 실물에서 자동적으로 나온다는 성질은 같다.

카탈로그를 도입할 때 도구만큼 중요한 결정이 “누가 채우고 유지하는가”다. 두 운영 모델이 있다. 중앙 집중형은 데이터 관리 부서 하나가 모든 피처의 메타를 등록·검수한다 — 표준은 일관되지만 병목이 되기 쉽고, 정의를 가장 잘 아는 생산 부서와 등록 주체가 서로 다르다. 연합형(federated, 데이터 메시(data mesh)의 발상)은 피처를 만든 부서가 자기 피처의 카탈로그 항목을 소유·유지하고, 중앙은 표준과 게이트만 정한다 — 소유와 지식이 일치하지만 부서마다 등록 품질이 일관되지 않을 위험이 있다. 실습이 정의를 생산 산출물(8장 registry 추출 JSON)에서 자동으로 끌어온 것은 연합형의 정신에 가깝다 — 정의의 소유권을 생산 지점에 두되, 등록은 수작업이 아니라 실물 추출로 표준화했다. 공공기관처럼 부서 자율성이 크고 데이터가 부서별로 흩어진 조직에서는 연합형이 현실적이되, 중앙이 최소 스키마(소유자·목적·개인정보 등급·소비자는 필수)를 게이트로 강제해야 등록 품질이 저하되지 않는다.

민간에서는 같은 등록소를 상용 데이터 카탈로그 플랫폼(Colibra, Atlan, Alation 등)이 담당하고, DataHub·OpenLineage는 오픈소스 쪽 선택지다. 스키마는 거의 같은데 도입 이유의 순서가 뒤집힌다 — 공공은 감사 증빙과 개인정보 보호법 대응을 먼저 놓고 재사용 효율을 뒤에 놓지만, 민간은 데이터 발견(discovery)과 중복 구현 제거를 먼저 놓는다. 두 목적에서 관리하는 지표도 다르다. 민간에서 카탈로그의 성숙도를 측정하는 대표 지표가 **데이터 자산 등록률** — 전체 피처·데이터셋 중 소유자·스키마·소비자가 채워진 항목의 비율 — 인데, 등록되지 않은 자산(shadow dataset)은 품질·접근·변경 통제 밖에 놓이므로 이 비율이 곧 통제 가능 범위다. 공공에서 이 지표를 그대로 쓸 수 있고, 다만 분모에 “개인정보 등급이 비어 있는 항목”을 별도로 세어 두면 감사 점검표 6·7번(표 13.6)의 입력이 된다. 계보(lineage) 도구의

쓰임새도 도메인에 따라 달라진다 — 공공은 “이 판단의 근거가 어디서 왔는가”를 감사에서 추적하는 데 쓰고, 민간은 장애 원인 분석과 규제 보고(금융권의 BCBS 239 등)에 쓴다. 추적하는 그래프는 같고, 그 그래프를 조회하는 상황이 다르다.

워크드 예제: 8장 정의에서 카탈로그를 생성

실습에서는 카탈로그를 수동으로 작성하지 않는다. 8장이 registry에서 추출해 커밋한

ch7_feature_definitions.json 을 읽어, 그 안의 정의·소유·개인정보 항목을 그대로 카탈로그 행으로 옮긴다. 여기에 저장소에 문서화된 사실로부터 **소비자** 컬럼을 채운다 — 이 부분이 8장 정의서에는 없던 거버넌스의 추가분이다.

표 13.2 피쳐 카탈로그(실측 — ch12_governance_report.json)

피쳐(dtype)	정의	소유자	개인정보	소비자	버전
complaint_count(Int64)	전일 확정 민원 건수 (7장 일별 집계)	민원데이터팀 (가상)	없음 (지역 단위 집계)	9장 훈련·10장 서빙 ·14장 통합(3)	complaint_model_v1
day_unmapped_rate(Float64)	전일 정제 품질 — 매핑 실패율	민원데이터팀 (가상)	없음 (지역 단위 집계)	complaint_model_v1 서비스(1)	complaint_model_v1

엔티티는 district(lawd_cd) — 지역명이 아니라 법정동 시군구 코드다. 8.4에서 강조한 표준 코드 원칙 (이름이 아니라 코드로 불린다)이 카탈로그의 조인 키에서도 반복된다. 소비자 컬럼의 수치에 주목한다 — complaint_count 에는 훈련·서빙·통합 세 소비자가 있으므로 변경 시 영향 범위가 넓고, day_unmapped_rate 는 하나다. 이 숫자가 13.4 변경 관리의 영향 분석에 그대로 쓰인다.

complaint_count 를 기준으로 카탈로그 한 행의 구성 과정을 확인한다. 정의·자료형·소유자·개인정보는 ch7_feature_definitions.json 의 대응 필드에서 그대로 오며(description → 정의, dtype → 자료형, governance.owner → 소유자, governance.privacy → 개인정보), 타임스탬프 의미(“값이 확정·공개된 시점”)와 TTL(3일), 버전(complaint_model_v1)도 8장 정의에서 복사된다. 유일하게 8장에 없던 것은 **소비자** 컬럼이며, 이것은 저장소에 문서화된 장 간 사실에서 구성한다 — 9장이 이 피쳐로 훈련 쌍을 만들었고, 10장 champion 서빙이 이 값을 입력으로 받으며, 14장 통합 MVP가 같은 확정 집계를 소비한다. 즉 카탈로그 행 = 8장 정의(복사) + 소비 사실(장 간 연결). 어느 쪽도 수작업으로 지어내지 않았다는 것이 이 실습의 하드를 준수 지점이다.

축소 지점: 카탈로그가 2행뿐이라는 것

실측 카탈로그의 피처는 2종이다. 빈약해 보인다면 그것이 요점이다 — 8장이 “피처 스토어의 무게는 계산이 아니라 규율에 있다”고 했듯, 카탈로그의 가치는 항목 수가 아니라 **모든 항목이 소유자·목적·개인정보·소비자를 빠짐없이 갖는 구조**에 있다. 피처가 수백 개가 되어도 카탈로그의 스키마는 같고, 오히려 그때 카탈로그 없이 “누가 무엇을 쓰는지”를 사람 기억에 의존하는 것이 불가능해진다. 실습이 2종으로 구조를 보여 주는 이유는, 규율은 규모와 무관하게 같은 형태이기 때문이다.

공공 사례

여러 부서가 각자 만든 피처를 공유하는 기관 데이터 플랫폼을 가정한다. 카탈로그가 없으면 A부서는 B부서가 이미 만든 “지역별 민원 건수”를 모른 채 다시 만들고(중복 구현 → 스큐), C부서는 그 피처가 개인정보를 포함하는지 모른 채 소비한다. 카탈로그는 “이미 있는가, 누구 것인가, 개인정보인가”를 한 번의 조회로 답한다 — 검수 질문은 “재사용한다는 피처가 카탈로그에 등록되어 있고 소비자 목록이 유지되는가”다. ### 핵심 정리 - 카탈로그는 피처의 주소록 — 정의·소유·원천·개인정보·소비자·버전을 한곳에 모은다. - 카탈로그는 실제 정의·소비 사실에서 생성하며 수동으로 작성하지 않는다. - 소비자 컬럼은 변경 영향 분석의 근거이며, 카탈로그의 가치는 항목 수가 아니라 구조에 있다.

13.3 접근 제어와 사용 이력

학습 목표

역할 기반 접근 제어(RBAC)와 목적 제한을 구분한다.

접근 로그가 감사에서 하는 역할과, 로그에서 점검표를 자동 생성하는 방법을 이해한다.

접근 통제의 두 축: 인가와 사용 이력

접근 통제가 성립하려면 두 장치가 함께 있어야 한다 — 인가(누가 어떤 행위를 할 수 있는가)와 사용 이력(누가 언제 무엇을 조회했는가)이다. 인가만 있고 이력이 없으면 사후에 “누가 그 값을 봤나”에 답할 수 없고, 이력만 있고 인가가 없으면 아무나 접근한 뒤 기록만 남는다. 내부 직원과 외부 감사자는 같은 피처라도 조회할 수 있는 범위가 다르다 — 감사자는 접근 기록은 보되 원시 값은 조회하지 않는다. 피처 접근 제어가 이 구조를 그대로 따른다: 역할×행위가 인가, `access_log` 가 사용 이력, 역할마다 다른 조회 범위가 목적 제한이다.

핵심 개념: 누가·무엇을·왜 — 그리고 전부 기록

접근 제어는 두 가지를 분리한다. **인가(authorization)**는 “이 역할이 이 행위를 해도 되는가”이고, **사용 이력(audit log)**은 “실제로 누가 언제 무엇을 했는가”다. 인가만 있고 이력이 없으면 사후에 “누가 이 피처를 봤는가”에 답할 수 없고, 이력만 있고 인가가 없으면 아무나 접근한 뒤 기록만 남는다. 거버넌스는 둘 다 요구한다.

실습은 세 가지 행위를 구분한다 — `read_definition` (카탈로그 메타 열람), `read_value` (원시 피처 값 조회), `read_log` (사용 이력 열람). 그리고 다섯 역할에 허용 행위를 배정한다.

표 13.3 접근 정책 행렬(역할 × 행위)

역할	read_definition	read_value	read_log
training_pipeline(9장 훈련)	허용	허용	불가
serving_api(10장 서빙)	허용	허용	불가
analyst(분석가)	허용	허용	불가
external_auditor(외부감사)	허용	불가	허용
미등록 역할	불가	불가	불가

이 행렬에서 주목할 부분은 외부감사 행이다. 감사자는 정의와 로그는 보되 **원시 값은 조회하지 못한다**. 감사는 값이 맞는지가 아니라 절차와 기록이 지켜졌는지를 보기 때문이며, 값 접근을 목적에 맞게 최소화하는 것이 개인정보 보호법 제18조(목적 외 이용 제한)의 정신이다. “시스템에 접근 가능”이라는 포괄적인 권한이 아니라, 역할×행위 단위로 세분한 권한이 목적 제한을 강제한다.

실습은 역할 기반 접근 제어(RBAC)를 쓰지만, 목적 제한을 더 정밀하게 집행하려는 현장은 속성 기반(ABAC)이나 목적 기반(PBAC) 접근 제어로 나아간다. RBAC이 “역할이 무엇인가”만으로 판정한다면, ABAC은 요청의 속성 — 접근 목적, 피처의 민감도 등급, 시간대, 소속 부서 — 을 함께 평가한다. 목적 제한이 자연스럽게 속성 하나(“이 조회의 목적이 피처의 등록 목적과 일치하는가”)가 되는 것이 요점이다. 실습의 판정 로직이 역할·피처 등록·행위의 세 관문을 차례로 검사하는 구조는 이미 이 방향의 축소판이다 — 여기에 “목적 일치” 속성과 “소유자 승인 여부” 속성을 조건으로 더하면 2·3등급 피처로 자연스럽게 확장된다(보충의 승인 워크플로가 그 확장이다). 즉 RBAC은 시작점이지, 목적 제한의 종착점이 아니다.

워크드 예제: 12건의 요청, 8 허용·4 거부

실습은 결정적 순서의 접근 요청 12건을 정책으로 판정하고 전부 `access_log`에 적재했다(허용·거부 무관하게 기록). 결과는 허용 8건, 거부 4건이다. 거부 4건의 사유를 하나씩 확인한다 — 각 사유가 서로 다른 통제 지점을 드러낸다.

표 13.4 접근 거부 4건과 통제 지점(실측)

요청	거부 사유	드러나는 통제
analyst · complaint_count · read_log	로그 열람은 감사 권한 필요	행위 단위 권한 분리
external_auditor · complaint_count · read_value	감사 역할은 원시 값 조회 불가(목적 제한)	목적 제한 설계(개인정보 피처 확장 시 PIPA 제18조)
marketing_bot · complaint_count · read_value	미등록 역할 — 정책에 없음	기본 거부(deny by default)
analyst · resident_arrears · read_value	카탈로그 미등록 피처 — 접근의 전제는 등록	카탈로그 게이트

넷째 행이 특히 중요하다. `resident_arrears` (가구 단위 체납 같은 민감 피처를 가정한 이름)는 카탈로그에 없다 — 그래서 값이 있고 없고를 따지기 전에 요청 자체가 거부된다. 접근의 전제는 등록이며, 이것이 “카탈로그가 접근 제어의 관문”이라는 뜻이다. 등록되지 않은 피처는 조회 대상조차 아니므로, 카탈로그 밖에서 별도로 생성해 사용하는 피처(shadow feature)가 통제를 우회할 수 없다.

판정 엔진이 요청 하나를 어떻게 처리하는지 세 요청을 순서대로 확인한다. 순서가 곧 통제의 우선순위다.

`training_pipeline · complaint_count · read_value`: ① 역할이 정책에 있는가 — 있다. ② 피처가 카탈로그에 있는가 — 있다. ③ 이 역할에 이 행위 권한이 있는가 — `read_value` 가 허용 집합에 있다. → **ALLOW**.

`external_auditor · complaint_count · read_value`: ①·② 통과. ③ 감사 역할의 허용 집합은 `{read_definition, read_log}` 뿐 — `read_value` 가 없다. → **DENY**(감사 역할 값 조회 제한). 값이 존재하는데도 거부된다는 점이 핵심이다 — 목적 제한은 “값이 없어서”가 아니라 “조회할 근거가 없어서” 차단한다.

`analyst · resident_arrears · read_value`: ① 통과. ② 피처가 카탈로그에 없다 → 즉시 **DENY**. 역할 권한을 따지기도 전에 등록 여부에서 걸린다.

세 판정의 순서는 다음과 같다 — 역할 등록 → 피처 등록 → 행위 권한. 이 순서가 뒤집히면(예: 행위 권한을 먼저 보면) 카탈로그에 없는 피처에 대한 권한을 논하게 되는 모순이 생긴다. 기본값은 거부이고, 허용은 세 관문을 모두 통과한 경우뿐이다.

접근 등급은 개인정보 민감도에 따라 계층화한다. 실습 피처는 모두 1등급(공개 집계)이지만, 상위 등급이 존재하는 순간 “시스템 접근 가능”이라는 포괄적인 권한은 무의미해진다 — 8.4의 등급 설계(표 8.5)를 접근 통제 관점으로 확장하면 다음과 같다.

표 13.5 접근 등급별 통제 설계(8.5 확장 — 값은 예시 설계, 실습 피처는 전부 1등급)

등급	피처 성격	조회 허용 역할	값 조회 통제	법적 근거
1. 공개 집계	지역·기관 단위 통계	전 부서 모델·분석가	로그만	공공데이터 개방
2. 내부 업무	업무 민감(비개인)	소관 부서 승인 모델	소유자 승인 + 로그	기관 내부 규정
3. 개인정보	개인·가구 단위	법적 근거 있는 모델만	목적 제한·보존 기한·감사	개인정보 보호법 제18조(목적 외 이용 제한)
4. 민감정보	건강·성생활 등	별도 동의·법령 근거 모델	처리 제한·별도 동의·안전조치	개인정보 보호법 제23조(민감정보 처리 제한)

두 상위 등급의 법적 근거를 구분해야 한다 — 개인·가구 단위 개인정보는 등록 목적 밖 이용·제공이 개인정보 보호법 제18조(목적 외 이용·제공 제한)로 제한되고, 그중 건강·성생활 등 **민감정보**는 제23조(민감정보 처리 제한)에 따라 별도 동의·처리 제한·안전조치가 더해진다. 민감정보는 개인정보의 부분집합이므로 제23조를 개인정보 전체의 근거로 넓혀 인용하지 않는다. 등급이 시스템 단위가 아니라 **피처 단위**로 붙는다는 것이 요점이다 — 상위 등급 피처 하나의 존재가 “피처 스토어에 접근 가능”이라는 포괄적인 권한을 무의미하게 만든다.

등급을 몇 단계로 나눌 것인가

표 13.5의 4단계는 정답이 아니라 하나의 선택이다. 등급 수를 정할 때 판단해야 할 것은 두 가지다. 첫째, **통제가 실제로 달라지는 지점마다 한 단계씩** 둔다. 2단계와 3단계를 합치면 “소유자 승인”과 “법적 근거 확인”이 한 칸에 들어가고, 그러면 승인만 받고 법적 근거 없이 개인정보 피처를 여는 판정이 정책상 허용된다. 반대로 통제가 같은 두 등급을 나누면 등록 시점에 어느 쪽인지 고민만 늘고 집행은 동일하다 — 등급 하나를 늘리는 비용은 정책 표 한 줄이 아니라 모든 피처 등록자의 판단 부담이다. 둘째, **각 등급의 법적 근거가 서로 다른 조문을 가리키는가**를 확인한다. 표 13.5에서 3등급(개인정보 보호법 제18조)과 4등급(제23조)을 나눈 이유가 이것이다 — 민감정보는 별도 동의와 안전조치가 추가로 붙으므로 3등급의 통제로는 부족하다. 근거 조문이 같은데 등급만 나뉘어 있다면 그 경계는 집행되지 않는다.

등급을 구분하는 **기준도** 선택 대상이다. 실습은 개인정보 민감도 하나를 축으로 썼지만, 현장에서는 데이터 주체의 식별 가능성(집계인가 개인 단위인가), 유출 시 피해의 회복 가능성(정정 가능한 정보인가 회복 불가능한 정보인가), 법적 근거의 종류(공개 의무인가 동의 기반인가) 중 어느 것을 1축으로 삼느냐에 따라 같은 피처가 다른 등급을 받는다. 축을 둘 이상 쓰면 등급이 행렬이 되어 집행이 복잡해지므로, 1축을 정하고 나머지는 등급 안의 추가 조건으로 붙이는 편이 낫다. 어느 축을 골랐든 등록 담당자가 새 피처를 보고 5초 안에 등급을 고를 수 있어야 하며, 고민이 길어지는 피처가 반복해서 나온다면 그것이 축을 잘못 선택했다는 신호다.

접근 기록은 규범이 요구하는 통제다

접근 로그를 남기는 것은 좋은 습관에 그치지 않고 규범이 요구하는 의무다. 개인정보보호위원회의 「개인정보의 안전성 확보조치 기준」(고시)은 안전조치로 **접근 권한의 관리**(제5조 — 역할별로 필요한 만큼만 부여)와 **접속기록의 보관 및 점검**(제8조 — 누가 언제 무엇에 접속했는지 기록하고 주기적으로 확인)을 요구한다 — 실습의 정책 행렬(역할×행위)이 전자, `access_log`가 후자에 해당한다. 이 규범은 피처가 개인정보를 담을 때 직접 적용되지만, 담지 않는 공개 집계 피처라도 “누가 왜 봤는가”를 기록해 두는 설계는 감사 대응 비용을 사후 조사에서 상시 기록으로 옮긴다는 점에서 같은 가치를 가진다 (안전조치 기준은 개인정보 처리를 규율하므로, 비개인 공개 집계 피처에는 직접 근거가 아니라 배경·일반 원칙으로 인용한다). 국제 프레임으로 보면 이것은 1장의 NIST AI RMF에서 GOVERN 기능이 요구하는 “기록과 책임 구조”의 데이터 층 집행이다 — 정책은 인가를, 로그는 책임 추적, 감사 점검표는 그 둘의 준수 증명을 담당한다.

민간에서 같은 두 장치는 신원·접근 관리(IAM, Identity and Access Management) 정책과 데이터 접근 감사 로그라는 이름으로 존재한다 — AWS Lake Formation의 테이블·컬럼 권한, Snowflake의 접근 정책, Databricks Unity Catalog가 실습의 역할×행위 행렬을 대체한다. 구조는 같고 로그를 조회하는 상황이 다르다. 민간은 접근 로그를 주로 보안 사고 탐지와 규제 준수(compliance) 감사에 쓰고, 공공은 정보공개 청구와 개인정보 감사에 쓴다. 목적 제한도 마찬가지로 이름만 바뀐다 — 개인정보 보호법 제18조에 대응하는 것이 EU GDPR 제5조의 목적 제한 원칙과 CCPA의 수집 목적 외 사용 금지이며, GDPR 적용 기업은 처리활동 기록(ROPA, Record of Processing Activities)으로 공공과 사실상 같은 수준의 기록 의무를 진다. 다만 위반의 대가가 다르다 — 공공은 법령 위반에 행정 제재가 바로 따르고, 민간은 과징금(GDPR은 전 세계 연 매출의 최대 4%)과 브랜드 신뢰 훼손이 앞선다. 한 가지 실질적 차이는 **거부 로그**다. 실습이 거부 4건을 허용 8건과 같은 무게로 기록한 것은 감사가 “막았는가”를 묻기 때문인데, 민간에서는 성능·수익 분석 로그가 전면에 놓이기 쉬워 거부 기록이 후순위가 되는 경우가 있다. 그래서 민간 쪽에서

이 통제의 성숙도를 측정하는 지표가 **접근 정책 준수율** — 전체 접근 요청 중 정책으로 판정되고 로그로 남은 비율 —이며, 허용·거부를 가리지 않고 기록되어야 이 값이 100%에 닿는다. 실습의 12건은 전부 판정·기록되었으므로 이 지표로는 12/12다(감사 점검표 1.2번이 그 확인이다).

감사 점검표는 로그에서 자동 생성한다

거부 요청이 정책과 일치하는지, 값 조회가 모두 인가된 역할이었는지를 사람이 직접 확인하면 누락과 왜곡이 발생한다. 실습은 14장의 인수 검수표 자동 생성 선례를 따라, `access_log`·카탈로그·품질·변경 이력에서 **10개 항목의 감사 점검표를 자동 생성했다** — 사람의 주장이 아니라 기록이 근거다. 실측 결과 10개 항목 전항 통과했다.

표 13.6 감사 점검표(자동 생성, `ch12_governance_checklist.json` — 전 10항목)

번호	점검 항목	판정
1	모든 접근 요청이 로그에 기록되었는가	통과
2	거부된 요청이 모두 사유를 보유하는가	통과
3	허용된 원시 값 조회가 모두 인가된 역할인가	통과
4	미등록 역할의 접근이 모두 거부되었는가	통과
5	감사 역할의 원시 값 조회가 0건인가(목적 제한)	통과
6	카탈로그의 모든 피처가 소유자·개인정보 등급·소비자를 보유하는가	통과
7	개인정보 등급이 비어 있는 피처가 0건인가	통과
8	품질 규칙(완전성·정확성·일관성) 3종이 모두 통과했는가	통과
9	변경 이력에 요청자·승인자·사유가 기록되었는가	통과
10	정의 변경이 영향 소비자를 식별했는가	통과

열 항목이 각기 다른 기록을 근거로 삼는다는 점에 주목한다 — 1.2번은 로그의 완전성, 3~5번은 접근 판정의 정합성, 6·7번은 카탈로그 메타의 완비, 8번은 품질, 9·10번은 변경 이력이다. 감사가 이 표 하나로 압축되는 것은 각 항목이 실물 기록에서 자동 계산되기 때문이며, 사람이 “지켰다”고 주장하는 자리가 없다.

공공 사례

정보공개 청구나 개인정보 감사에서 “이 데이터를 누가 언제 조회했는가”라는 질문이 온다. 각자 계산 구조에서는 각 시스템의 로그를 뒤져 재구성해야 하고, 그마저 남아 있지 않으면 답할 수 없다. 접근 로그가 피쳐 조회의 단일 지점에 있으면 질의 하나로 답한다 — 감사 대응 비용이 사후 조사에서 상시 기록으로 옮겨 간다. 검수 질문: 피쳐 조회가 로그로 남는가, 그리고 그 로그에서 감사 점검표가 자동으로 나오는가. ### 핵심 정리 - 접근 제어는 인가(역할×행위)와 사용 이력(로그)을 분리해 둘 다 요구한다. - 목적 제한은 뭉뚱그린 권한이 아니라 역할×행위 단위 권한으로 강제된다(감사자 값 조회 불가). - 감사 점검표는 로그에서 자동 생성한다 — 실측 12건 판정, 점검 10항목 전항 통과. - 등급 수는 통제가 실제로 달라지는 지점만큼 두고, 등급을 구분하는 축은 하나만 고른다. - 민간의 IAM 정책과 처리활동 기록이 같은 구조를 담지만, 거부 로그를 감사 증거로 명시 관리하는 것은 공공 쪽 요구다.

13.4 품질 점검과 변경 관리

학습 목표

피쳐 단위 품질 규칙(계약)과 11장 분포 감시(드리프트)를 구분한다.
정의 변경의 영향 소비자를 식별하고 승인 기록을 남기는 이유를 이해한다.

품질 규칙은 게이트, 드리프트는 감시

11장은 분포 $P(X)$ 가 시간이 지나며 이동하는지를 감시했다(경보 ≠ 즉시 조치). 13.4의 품질 점검은 다른 층이다 — 개별 피쳐 값이 **계약을 위반했는가**를 배포·서빙 전에 막는 게이트다. 드리프트는 “환경 변화의 원인을 확인해야 한다”는 신호이고, 품질 위반은 “이 값은 애초에 틀렸으니 쓰면 안 된다”는 차단이다. 둘을 뭉치면, 정상적인 계절 변동을 데이터 오류로 오인하거나 반대로 명백한 결측을 분포 감시가 놓친다.

표 13.7 품질 게이트와 드리프트 감시의 층 구분

구분	품질 게이트(13.4)	드리프트 감시(11장)
질문	이 값이 계약을 위반했는가	분포가 이동했는가
판정 대상	개별 피쳐 값	값의 분포
위반 시	차단(배포·서빙 중단)	경보(원인 분석 후 판단)
예	음수 건수·중복 키·결측	계절적 민원 증가
오작동 위험	정상 변동을 오류로 차단	명백한 결측을 못 잡음

실습은 8장 실데이터 9행(3일 × 3지역)에 세 규칙을 적용했고, 공공데이터 품질관리의 3대 품질지표(정확성·완전성·일관성)에 맞췄다.

표 13.8 피쳐 품질 규칙 판정(실측 9행)

규칙	공공 품질지표	검사 내용	판정
결측 없음	완전성	complaint_count·day_unmapped_rate 값 존재	9/9 통과
값 범위	정확성	건수 ≥ 0 , 실패율 $\in [0,1]$	9/9 통과
키 유일성	일관성	(lawd_cd, event_timestamp) 중복 없음	9/9 통과

세 규칙 모두 통과한 것은 8장 확정 산출물이 이미 정제·검증을 거쳤기 때문이다(7장 품질 게이트 PASS).
요점은 통과 자체가 아니라, 이 규칙들이 **배포 전에 자동으로 돌아 위반을 차단하는 지점**이 있다는
것이다 — 규칙이 없으면 음수 건수나 중복 키가 조용히 모델에 들어간다(8장 침묵 실패 목록의 연장).

규칙이 없었다면 무슨 일이 생겼을지 반사실로 설명한다. 만약 어느 날 정제 파이프라인의 버그로 강남구
행이 두 번 적재되어 (11680, 2026-07-04 00:00) 키가 2행이 되었다면 — 유일성 규칙이 없으면 이
중복은 시점 조인에서 같은 키에 두 값을 만들고, 훈련 셋의 그 지역 가중치를 조용히 두 배로 만든다.
유일성 규칙(일관성)은 이 중복을 배포 전에 검출해 차단한다. 마찬가지로 실패율이 계산 오류로 1.2가
되었다면 범위 규칙(정확성)이, 건수가 누락되어 null이 되었다면 결측 규칙(완전성)이 각각 다른 실패를
막는다. 세 규칙은 서로 다른 사고를 담당하며, 어느 하나만 빠져도 그 유형의 사고가 통과한다 — 품질
게이트가 규칙의 집합인 이유다.

임계값은 어디서 나오는가

표 13.8의 세 규칙은 임계값이 0이라 고를 것이 없어 보인다 — 결측은 한 건도 안 되고, 중복 키도 한
건도 안 되며, 실패율은 $[0,1]$ 을 벗어나면 안 된다. 이 세 값이 0인 것은 게으름이 아니라 규칙의 성격이다.
결측·중복·범위 이탈은 정도의 문제가 아니라 계약 위반 여부의 문제라, 허용 건수를 1건이라도 열면 “몇
건까지 괜찮은가”라는 답 없는 협상이 시작된다. 그런데 규칙을 하나만 더 추가하면 곧바로 고를 값이
생긴다. 예컨대 “매핑 실패율이 X를 넘으면 배포를 막는다”에서 X는 0이 될 수 없다 — 실패율 0을
요구하면 원천 코드 체계가 바뀌는 날마다 배포가 중단된다.

이때 X를 무엇에서 역산하는지가 판단이다. 디스크나 관례가 아니라 **하류에서 견딜 수 있는 오차**에서
역산한다. 매핑 실패는 그 지역의 민원이 집계에서 빠진다는 뜻이므로, “인력 배치 산정이 몇 %까지
틀려도 배치 결과가 안 바뀌는가”가 상한을 정한다. 이 질문에 답할 수 있는 사람은 데이터 담당자가
아니라 배치를 결정하는 부서이며, 그래서 임계값 결정은 소유자(13.1)의 책임 범위에 들어간다. 표
13.9의 실측 3일분에 상한을 적용하면 그 폭이 드러난다. 실패율은 0.05-0.0588-0.0455이고, 상한을 6%로
설정하면 세 날 모두 통과하고, 5%로 설정하면 2026-07-02 하루만 차단되며, 4.5%로 설정하면 사흘 모두
차단된다. 즉 1.5%포인트 차이가 “전부 통과”와 “전부 차단”을 결정한다. 임계값을 관례로 고르면 이
민감도를 모른 채 고르는 것이다. 그러면 게이트가 매일 발동해 무시되거나, 한 번도 발동하지 않아 없는
것과 같아진다.

민간에서 같은 게이트를 세우는 도구가 Great Expectations, Soda, dbt tests다 — 파이프라인 중간에
계약 위반을 차단하는 위치도, 규칙을 코드로 두고 CI에서 돌리는 방식도 같다. 다른 것은 규칙이 어디서
오는지다. 공공은 품질관리 3대 지표(정확성·완전성·일관성)라는 행정 규범이 규칙의 범주를 먼저 정해
주지만, 민간은 비즈니스 규칙(주문 금액 음수 불가, 고객 ID 중복 불가)에서 규칙을 직접 도출하고 위반의
대가를 매출·고객 경험으로 환산한다. 두 경우 모두 관리 지표는 **품질 게이트 통과율** — 규칙을 통과한

배치의 비율 —이며, 읽는 법도 같다. 통과율이 낮으면 원천 수집 단계에 문제가 있다는 신호이고, 통과율이 높은데도 하류 모델이 오동작하면 규칙 자체가 잘못 걸려 있다는 신호다. 실습 9행이 3규칙 모두 9/9로 통과한 것은 전자의 뜻이지 후자의 안전 보장이 아니다.

직관: 정의를 덮어쓰면 과거가 다시 쓰인다

피쳐 정의를 조용히 덮어쓰면, 같은 이름 아래 과거에 기록된 값과 현재 기록되는 값의 의미가 달라진다 — 정의 변경 이전에 계산된 값들이 소급으로 다른 산식의 결과로 뒤바뀌는 셈이다. 그 값을 사용한 모든 산출물이 두 정의에 걸쳐 뒤섞인다. 피쳐도 마찬가지다 — 같은 이름 아래 값의 의미가 달라지면, 그 피쳐로 훈련된 모델은 어느 정의의 값으로 배웠는지 모른 채 다른 정의의 값으로 서빙받는다(8.4의 역설, 훈련-서빙 스큐의 재발). 그래서 정의 변경은 덮어쓰기가 아니라 버전 발행이어야 한다.

정의를 계약으로: 데이터 계약 명세

1장에서 데이터 계약(data contract)을 “생산자와 소비자가 스키마·의미·품질·변경 절차를 합의해 문서와 코드로 관리하는 약속”으로 정의했다. 피쳐 정의는 그 계약의 한 항목이고, 최근에는 이 약속을 사람이 읽는 문서가 아니라 기계가 읽는 명세로 형식화하는 흐름이 자리 잡았다. 대표적으로 데이터 계약 명세(Data Contract Specification)는 스키마·의미·품질 규칙·변경 정책을 하나의 기계가독 파일로 적는 오픈 명세다. 계약을 이렇게 코드로 두면(1장의 “계약을 코드로 검증한다”의 연장) 계약 위반을 CI에서 자동 검사할 수 있고, 소비자 목록을 함께 담으면 변경 통보의 대상도 명세에서 나온다. 실습의 카탈로그 한 행 — 정의·소유자·목적·개인정보·소비자·버전·TTL —이 바로 이 계약의 최소 형태다. 카탈로그를 “기계가 읽어 검사할 수 있는 계약”으로 보면, 13.4의 품질 규칙(게이트)과 변경 관리가 왜 같은 카탈로그에 붙는지가 분명해진다 — 품질 규칙과 소비자 목록이 계약의 조항이라면, 게이트와 통보는 그 조항의 집행이다. (데이터 계약 명세는 이 장의 직접 선례가 아니라 1장 개념의 현행 형식화이므로 방법론 차용 자격으로 인용한다.)

변경 관리: 정의가 바뀌면 값이 바뀐다

피쳐 정의는 불변이 아니다. 정제 규칙이 바뀌면 같은 이름의 피쳐가 다른 값을 갖고, 그 피쳐로 훈련된 모델의 훈련-서빙 일치가 소급으로 깨진다(8.4의 역설). 이것이 Sculley et al.(2015)이 말한 CACE(Changing Anything Changes Everything)의 피쳐판이다. 규율은 두 가지다 — 변경을 버전으로 내고(정의를 덮어쓰지 않는다), 변경 전에 **영향 소비자를 식별**한다.

실습은 `day_unmapped_rate`의 산식 변경을 실측했다. 현행 정의(v1)는 “미매핑 건수 ÷ 유효 건수(valid)”이고, 제안 정의(v2)는 “미매핑 건수 ÷ 물리 건수(physical)”다. 7장 실측에서 중복 제거(`dup_removed`)가 있었던 날만 유효 건수와 물리 건수가 다르므로, 정의 변경은 그 날의 값만 바꾼다.

표 13.9 변경 관리 실측 — `day_unmapped_rate` v1 → v2

집계일	미매핑	유효(valid)	물리(physical)	v1(÷valid)	v2(÷physical)	변경
2026-07-01	1	20	20	0.05	0.05	—
2026-07-02	1	17	18	0.0588	0.0556	변경
2026-07-03	1	22	22	0.0455	0.0455	—

2026-07-02 하루만 값이 변한다 — 그날만 중복 1건이 제거되어 유효(17)와 물리(18)가 갈리기 때문이다. 수작업으로 확인하면 $v1=1/17=0.0588$, $v2=1/18=0.0556$ 이다(8장이 기록한 0.059는 이 $v1$ 을 소수 셋째 자리로 반올림한 값이다). 정의 변경이 9행 중 3행(그날의 세 지역)에 영향을 준다. 영향 소비자는 카탈로그의 소비자 컬럼(표 13.2)에서 자동으로 나온다 — `day_unmapped_rate`의 소비자는 `complaint_model_v1` 서비스 다. 실습은 이 변경을 요청자·승인자·사유·영향 소비자·영향 행 수와 함께 `change_log`에 적재했다.

변경을 사후에 증명하는 장치가 9장에 이미 있었다. 9장은 훈련 데이터 6쌍에 sha256 지문 `96f6b6b3ce9dcdd1...`을 걸어 모든 실험의 파라미터에 넣었다 — “이 모델은 어느 데이터로 배웠는가”를 증명 가능하게 만든 것이다. 피쳐 정의가 바뀌면 그 피쳐로 만든 훈련 데이터의 지문도 바뀐다. 즉 변경 관리와 9장의 지문은 한 쌍이다. 카탈로그의 소비자 목록이 “누가 영향받는가”를 사전에 알린다면, 데이터 지문은 “실제로 어느 정의의 데이터로 훈련되었는가”를 사후에 대조한다. 감사에서 “이 모델이 개정 전 정의로 훈련되지 않았는가”라는 질문에, 지문 하나로 답할 수 있다.

한 가지 단서를 붙인다: 9장의 `baseline-linear` 모델은 실제 훈련에 건수만 썼으므로, 이 산식 변경이 그 두 모델의 수치를 당장 바꾸지는 않는다. 그런데도 영향 소비자로 `complaint_model_v1` 서비스가 잡히는 이유는, 카탈로그가 **서비스 등록 단위**로 소비자를 추적하기 때문이다. 필드 단위의 실제 사용 범위는 이보다 좁을 수 있고, 그 정밀한 추적이 계보(lineage) 도구가 필요한 지점이다(보충). 거버넌스의 기본 원칙은 “넓게 알리고 좁게 확인”이다 — 등록 소비자 전부에게 알린 뒤, 각 소비자가 자신이 실제로 그 필드를 쓰는지 판단한다.

통보 대상을 어디까지 잡을 것인가도 설계 결정이다. 소비자 목록을 서비스 단위로 두면 통보가 넓어지고 (실제로 안 쓰는 소비자에게도 간다), 필드 단위로 두면 정확해지는 대신 계보 도구가 필요하다. 여기서 두 오류의 대가가 비대칭이라는 점이 선택을 정한다 — 사용하지 않는 소비자에게 통보하면 그 부서가 한 번 확인하고 끝이지만, 사용하는 소비자를 누락하면 그 모델은 바뀐 정의를 모른 채 계속 운영되고 그 사실이 드러나는 것은 지표가 급변한 뒤다. 대가가 비대칭이면 넓은 쪽으로 기울이는 것이 맞고, 계보 도구를 도입한 뒤에야 좁힌다. 다만 넓은 통보를 반복하면 수신 부서가 통보를 무시하게 되므로, 통보문에 “영향 행 수와 영향 소비자”를 함께 실어 수신자가 자기 소관인지 즉시 판단할 수 있게 해야 한다 — 실습이 `change_log`에 요청자·승인자·사유·영향 소비자·영향 행 수를 함께 적재하는 이유가 이것이다.

민간에서는 같은 규율이 스키마 레지스트리(Schema Registry)와 데이터 계약으로 집행된다 — 스키마 변경의 호환성을 등록 시점에 검사하고, 하위 호환이 깨지면 배포를 막는다. 공공은 여기에 승인 기록을 감사 증빙으로 남겨야 하고, 민간은 호환성 검사를 CI/CD 파이프라인에서 자동화한다. 강제력의 출처가 다르다 — 공공은 감사가 사후에 묻고, 민간은 하위 호환이 깨지는 순간 추천 모델 오동작이나 결제 실패로 즉시 청구된다. 양쪽이 공통으로 쓰는 지표가 **스키마 변경 사전 통보율** — 영향받는 하류 소비자에게 변경 전에 통보가 간 변경의 비율 — 이며, 이 값을 세려면 소비자 목록이 유지되고 있어야 하므로 이 지표는 카탈로그 등록률과 한 쌍이다. 소비자 목록이 비어 있는 피쳐는 사전 통보율의 분모에조차 들어가지 못한다.

변경의 극단: 피쳐 폐기

변경 관리에서 가장 되돌리기 어려운 경우는 정의를 바꾸는 것이 아니라 피쳐를 아예 없애는 폐기(deprecation)다. 원칙은 단순하다 — **소비자가 없는 피쳐만 안전하게 폐기한다**. 카탈로그의 소비자 컬럼이 그 판정의 근거다. 실습 카탈로그로 직접 검산한다. `complaint_count`는 소비자가 셋(9장 훈련·10장 서빙·14장 통합)이므로 지금 폐기하면 세 소비자가 동시에 실패한다 — 폐기 불가이며, 폐기하려면

먼저 세 소비자를 다른 피처로 이전시켜 소비자 목록을 비워야 한다. 반면 소비자가 하나인 피처라면 그 하나만 이전하면 된다. 소비자 목록은 변경 통보의 수신자이며 폐기 가능 여부를 판정하는 근거다. 카탈로그가 없으면 이 판정을 “아무도 사용하지 않을 것”이라는 추정으로 대체하게 되고, 그 추정이 틀리는 순간이 8장의 침묵 실패로 되돌아간다 — 사용되던 피처가 조용히 삭제되고, 그것을 읽던 조인이 기록 없이 빈 값을 반환한다. 등록된 소비자가 없을 때만 피처를 폐기할 수 있다.

공공 사례

민원 통계의 산정 방식을 개정하면(예: 중복 신고 처리 기준 변경), 그 통계를 피처로 쓰는 모든 모델이 영향을 받는다. 카탈로그의 소비자 목록이 없으면 개정 부서는 누구에게 알려야 할지 모르고, 소비 모델은 어느 날 갑자기 지표가 튀는 이유를 모른 채 모델을 의심한다(8.1 공공 사례의 반복). 변경 관리가 있으면 개정은 버전으로 나가고, 소비자 목록으로 통보되며, 변경 이력에 승인이 남는다 — 검수 질문은 “피처 정의 변경에 승인 기록과 영향 소비자 식별이 있는가”다.

역방향: 이 장의 거버넌스를 민간 현장에 옮기면

13.1부터 여기까지는 민간에서 규제 대응으로 먼저 성숙한 장치를 공공으로 옮겨 왔다. 반대 방향도 성립한다 — 이 장에서 만든 카탈로그·접근 로그·품질 게이트·변경 이력을 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

이커머스 추천 피처의 공유에서는 사용자 클릭·구매 이력에서 뽑은 피처(최근 7일 구매 횟수, 카테고리 선호도)를 추천·검색·광고 팀이 나눠 쓴다. 카탈로그 스키마는 그대로다 — 정의·소유 팀·소비 모델·버전. 소비자 컬럼이 하는 일도 그대로다. 한 팀이 산식을 바꿨을 때 카탈로그가 없으면 다른 팀 모델의 성능이 조용히 저하되고, 그 팀은 자기 모델을 의심한다. 13.4의 CACE가 도메인을 가리지 않는다는 뜻이다. 바뀌는 것은 통보를 강제하는 힘이다 — 공공에서는 승인 기록이 감사에서 요구되지만, 여기서는 통보 없는 변경이 다음 분기 전환을 하락으로 청구된다.

핀테크 신용평가 피처는 공공의 요구가 거의 그대로 재현되는 현장이다. 연체 이력·소득 추정치 같은 피처는 신용정보법과 리스크 데이터 집계 원칙(BCBS 239)에 따라 “이 지표가 어느 원천에서 어떤 변환을 거쳐 나왔는가”를 계보로 증명해야 하고, 접근 로그는 금융 감독 검사에서 제출 대상이다. 실습의 네 장치 중 어느 것도 뺄 수 없고, 오히려 서비스 단위 소비자 추적을 필드 단위 계보로 정밀화해야 한다.

다중 고객 SaaS의 테넌트 격리에서는 접근 제어의 축이 하나 바뀐다. 공공에서 역할×행위였던 행렬에 “어느 고객(tenant)의 데이터인가”가 축으로 더해지고, 고객 A의 피처가 고객 B의 모델에 섞이면 그것은 권한 위반이 아니라 데이터 유출 사고다. 집행 지점은 Snowflake의 행 접근 정책이나 Databricks Unity Catalog의 테이블·컬럼 단위 통제로 내려가지만, 판정 순서 — 등록 확인 → 범위 확인 → 행위 권한 — 는 실습의 세 관문과 같다. 광고 플랫폼도 여기에 붙는다. 타겟팅 피처마다 처리 목적·보존 기한·법적 근거(동의)를 붙이는 것은 표 13.5의 등급 설계를 개인정보 등급 분류로 옮긴 것이고, 사용자가 삭제를 요청하면 계보 그래프로 원천·파생을 추적해 지운다. 이 현장들이 관리하는 규제 지표가 삭제 요청(GDPR 제17조 잊힐 권리) 처리 소요 시간과 데이터 주체 접근 요청(DSAR, Data Subject Access Request) 응답 시간이며, 법정 기한(GDPR 30일, CCPA 45일)을 넘기면 과징금 위험이 생긴다. 이 두 지표를 지킬 수 있게 하는 것이 결국 카탈로그와 계보다 — 어느 피처에 그 사람의 데이터가 들어갔는지 모르면 30일 안에 지울 수 없다.

이 셋을 관통하는 것은 하나다. 무엇을 카탈로그에 등록할 것인가, 접근을 몇 등급으로 나눌 것인가, 품질 임계값을 무엇에서 역산할 것인가, 변경 통보를 어디까지 보낼 것인가 — 이 네 결정은 어느 현장에서도 똑같이 내려야 하고, 각 결정의 근거만 도메인에 따라 달라진다. 공공에서 그 근거가 감사·형평성·법적 의무라면, 민간에서는 매출·장애 비용·규제 과징금이다. 방향은 비대칭이다 — 민간 장치를 공공에 도입하면 거부 로그와 목적 대조와 자동 점검표라는 세 겹이 추가되지만, 공공 설계를 민간에 도입하면 그 층을 제외해도 손실이 없다. 거부 로그는 보안 사고 탐지 자산이 되고, 자동 감사 점검표는 규제 대응 시간을 줄인다.

핵심 정리

품질 규칙은 계약 위반을 배포 전에 막는 게이트, 드리프트는 분포 이동의 감시 — 다른 층이다.

정의 변경은 버전으로 내고, 변경 전에 카탈로그의 소비자 목록으로 영향을 식별한다.

실측: 산식 $v1 \rightarrow v2$ 가 중복 있던 하루(3행)만 바꾸며, 영향 소비자·승인이 변경 이력에 기록된다.

임계값은 관례가 아니라 하루가 건널 수 있는 오차에서 역산한다 — 실측 3일분 실패율에서 1.5% 포인트가 모두 통과와 모두 차단을 결정한다.

등록·등급·임계값·통보 범위라는 네 결정은 도메인과 무관하게 내려야 하고, 근거만 도메인에 따라 달라진다.

실습 13.1 피쳐 거버넌스 설계와 구현(★★★☆☆)

이 실습은 설계 → 실행 → 대조 → 위험 식별 네 단계로 진행한다. 순서에 의도가 있다. 이미 정해진 정책 행렬과 품질 규칙으로 코드부터 실행하면 왜 등급이 낮인지, 왜 결측 허용치가 0인지 묻지 않게 되고, 그러면 감사 점검표는 “실행 결과 전향 통과”로 남는다. 먼저 자기 판단으로 등급과 임계값을 정하고 판정 결과를 예측한 다음 실행 산출물과 대조하면, 어긋난 지점이 곧 자기 설계의 결함이 된다. 감사·점검표는 그 뒤 4단계에서 다룬다 — 설계 근거와 실행 증거가 모두 확보되어야 감사 질문에 답할 수 있기 때문이다.

실습 목표

요구사항에서 접근 등급 체계와 역할별 허용 행위를 스스로 정하고 근거를 적는다(설계).

품질 규칙의 임계값을 하루가 건널 수 있는 오차에서 역산해 정한다(설계).

8장 정의와 7장 집계를 입력으로 SQLite 카탈로그를 구축하고, 접근 정책으로 사용 이력을 적재한다(실행).

자기 정책의 판정 예측을 실행 산출물의 실측 판정과 대조하고, 어긋난 원인을 설명한다(대조).

품질 규칙 3종을 적용하고, 정의 변경의 영향을 식별하며, 감사 점검표를 로그에서 자동 생성한다(증거).

입력 데이터

카탈로그의 정의는 8장이 registry에서 추출해 커밋한 `ch7_feature_definitions.json` 을, 피쳐 값 9행은 7장 확정 집계 3일치를 스냅샷으로 동봉한 것이다(`data/input/`). 소유자·부서 표기는 8장과 동일하게 가상(시뮬레이션)이며, `resident_arrears` 처럼 카탈로그에 없는 피쳐 이름은 거부 시연을 위한 가상

이름이다(값을 만들지 않고 요청만 거부된다). 인용 수치는 모두 실제 실행 산출물 (ch12_governance_report.json)에서 산출된다.

1단계 설계 — 접근 등급과 품질 임계값을 먼저 정한다

코드를 실행하기 전에 정책을 확정한다. 아래 요구사항을 읽고 표 13.10과 표 13.11의 빈칸을 직접 채운다. 본문의 표 13.3·13.5·13.8은 완성된 예시 답안이므로 그대로 옮겨 적지 않는다 — 채운 뒤에 비교한다.

요구사항(가상 발주 조건) - 기관 피처 플랫폼에 세 부서가 피처를 올린다. 지역 단위 집계 피처, 부서 내부 업무 피처, 가구 단위 체납 피처가 섞여 있다. - 외부 감사자가 반기 1회 접근 기록을 열람한다. 감사 대상은 절차와 기록이지 값이 아니다. - 개인정보를 담은 피처는 소관 부서장의 승인 없이 값을 조회할 수 없다. 승인에는 만료 기한이 붙는다. - 매핑에 실패해 빠진 민원은 그 지역의 인력 배치 산정을 낮춘다. 품질 위반이 서빙까지 흘러가면 안 된다. - 부서마다 결측·이상값 판정 기준을 다르게 쓰고 있다. 플랫폼 차원에서 하나로 정해야 한다.

표 13.10 접근 정책 설계표 — 학습자 작성용

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
등급 단계 수		
등급을 구분하는 축(하나만 고른다)		
가구 단위 체납 피처의 등급		
그 등급의 값 조회 통제		
외부 감사 역할에 허용할 행위		
정책에 없는 역할의 기본 판정		
이 설계가 깨지는 조건		

표 13.11 품질 규칙 임계값 설계표 — 학습자 작성용

규칙	공공 품질지표	내가 정한 임계값	역산 근거(무엇이 건딜 수 있는 오차인가)	위반 시 조치(차단·경고)
결측	완전성			
값 범위	정확성			
키 유일성	일관성			
매핑 실패율 상한	정확성			
이 설계가 깨지는 조건	—			

작성할 때 지킬 것이 네 가지다. 첫째, 근거 칸에 “기본값이라서” 또는 “보통 그렇게 한다”를 쓰지 않는다. 둘째, 임계값은 관례가 아니라 **하류가 견딜 수 있는 오차**에서 역산한다(13.4 「임계값은 어디서 나오는가」) — 매핑 실패율이라면 “인력 배치 결과가 안 바뀌는 오차가 몇 %인가”에서 시작한다. 셋째, 등급 수를 늘릴 때마다 그 등급의 통제와 법적 근거가 바로 아래 등급과 실제로 다른지 확인한다. 넷째, “이 설계가 깨지는 조건”에는 자기 선택이 불리해지는 상황을 적는다 — 트레이드오프를 이해하고 선택했는지 확인하는 칸이다.

판정 예측: 실행 스크립트는 결정적 순서의 접근 요청 12건을 정책으로 판정한다. 매핑 실패율 상한은 스크립트가 실행하는 세 규칙에 없으므로, 3단계에서 표 13.9의 실측 3일분에 자기 상한을 수작업으로 적용해 본다. 실행하기 **전에** 아래를 자기 정책 기준으로 적어 둔다.

예측 항목	내 예측	실제(3단계에서 기입)
12건 중 거부 건수		
거부가 드러내는 서로 다른 통제 지점의 수		
표 13.11의 실패율 상한에 걸리는 날 수(3일 중)		

세 칸 모두 한 줄 근거를 함께 적는다. 특히 둘째 칸은 건수가 아니라 **종류**를 세는 칸이다 — 같은 사유로 거부된 요청이 열 건이어도 통제 지점은 하나다.

2단계 실행 — 카탈로그·접근·품질·변경을 한 번에 돌린다

Python 3.10 이상(이 실습은 3.13에서 검증)이면 되고 외부 인프라·의존성이 없다 — 카탈로그·로그는 SQLite, 나머지는 표준 라이브러리다.

```
cd practice/chapter12
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt # 외부 의존성 없음
python run_governance.py
```

스크립트는 다음 순서로 진행하며, 각 단계가 절 하나에 대응한다.

단계	내용	대응 절
1	8장 정의 + 7장 집계 로드 → 피쳐 원천 9행 재구성	13.1
2	카탈로그 구축(SQLite <code>feature_catalog</code>)	13.2
3	접근 요청 12건 정책 판정 → <code>access_log</code> 적재	13.3
4	품질 규칙 3종 적용(결측·범위·유일성)	13.4
5	정의 변경 v1→v2 영향 분석 → <code>change_log</code>	13.4
6	감사 점검표 자동 생성 + 증거 JSON 저장	13.3·13.4

핵심 코드

접근 판정은 미등록 역할, 카탈로그 미등록 피쳐, 권한 없는 행위를 차례로 거부하고 나머지를 허용한다.

```

if role not in ACCESS_POLICY:          decision, reason = "DENY", "미등록 역할"
elif feature not in catalog_names:      decision, reason = "DENY", "카탈로그 미등록 피처"
elif action not in ACCESS_POLICY[role]: decision, reason = "DENY", "권한 없음(목적 제한)"
else:                                  decision, reason = "ALLOW", "정책 허용"

```

전체 코드는 [practice/chapter12/code/12-1-feature-governance.py](#) 참고

결정성 설계

접근·변경 타임스탬프는 `now()` 가 아니라 **입력에서 유도**한다 — 마지막 피처 확정 시점(2026-07-04 00:00 UTC)을 기준으로 요청 순번만큼 분을 더한다(8장 `as_of` 방식). SQLite 파일은 빌드 산출물이라 커밋하지 않고(`.gitignore`), 커밋 증거는 JSON뿐이다. 그 결과 증거 JSON은 재실행해도 바이트 동일하다 — 두 번 실행해 `diff` 가 비어 있음을 확인했다.

실행 중 확인 체크리스트

- ☐ 카탈로그가 2피처(+엔티티 `district`)로 생성되었는가 — 수작업으로 작성한 항목이 없는가
- ☐ 접근 요청 12건 중 허용 8·거부 4이고, 거부 4건의 사유가 서로 다른가(표 13.4)
- ☐ 품질 규칙 3종이 9행에서 모두 통과(9/9)했는가
- ☐ 변경 관리에서 2026-07-02만 값이 바뀌고(0.0588→0.0556), 영향 행이 3인가
- ☐ 감사 점검표가 10항목 전항 통과하고 마지막 줄이 `CH13_RUN_PASS` 인가
- ☐ 증거 JSON을 재실행해도 바이트 동일한가
- ☐ 실행 전에 표 13.10·13.11과 판정 예측을 적어 두었는가(실행 후에 적으면 대조가 성립하지 않는다)

산출물 안내

산출물	내용	용도
<code>ch12_governance_report.json</code>	카탈로그·접근로그·품질·변경·감사 통합 증거	이 장 인용 수치의 증거 파일
<code>ch12_governance_checklist.json</code>	감사 점검표(자동 생성)	13.3 감사 자산 — 표 13.6의 원본
<code>ch12_governance_policy.md</code>	피처 거버넌스 정책 초안	실습 부속 정책 문서
<code>feature_governance.sqlite</code>	카탈로그·접근·변경 테이블(빌드 산출물)	커밋 제외 — 정의·로그의 실체

2단계 실행 결과(실제)

카탈로그: 피처 2종(`complaint_count·day_unmapped_rate`) + 엔티티 `district(lawd_cd)`, 소비자 3·1 — 표 13.2.

접근 제어: 요청 12건, 허용 8·거부 4. 거부는 로그 권한 없음·감사 값 조회 제한·미등록 역할·

카탈로그 미등록 각 1건 — 표 13.4.

품질: 결측·범위·유일성 3규칙 모두 9/9 통과 — 표 13.8.

변경 관리: `day_unmapped_rate v1→v2`가 2026-07-02만 변경(0.0588→0.0556), 영향 행 3, 영향 소비자 `complaint_model_v1` 서비스 — 표 13.9.

감사 점검표: 10항목 전항 통과, CH13_RUN_PASS. 증거 JSON 재실행 바이트 동일.

3단계 대조 — 설계 예측과 실행 결과의 대조

1단계 예측표의 “실제” 열을 ch12_governance_report.json 의 판정 결과로 채운다. 실측은 12건 중 거부 4건이고, 그 4건이 드러내는 통제 지점은 네 종류가 모두 다르다(표 13.4) — 행위 단위 권한 분리, 목적 제한, 기본 거부, 카탈로그 게이트다. 여기서 확인할 것 네 가지다.

거부 건수는 맞았는데 통제 지점 수를 적게 적었다면 무엇을 빠뜨렸는가. 정책의 강도는 몇 건을 막았는지가 아니라 서로 다른 이유로 막을 수 있는 관문이 몇 개인지로 정해진다. 관문이 셋뿐인 정책은 거부 건수가 같아도 네 번째 유형의 요청을 통과시킨다.

표 13.10에서 “정책에 없는 역할의 기본 판정”을 허용으로 적었다면 marketing_bot 의 요청이 통과한다. 등록되지 않은 역할이 값을 읽고, 그 사실은 로그에 “허용”으로 남는다. 기본값이 거부여야 하는 이유는 정책을 촘촘히 쓰기 위해서가 아니라, 정책을 다 못 썼을 때 안전한 쪽으로 실패하기 위해서다.

표 13.10에서 등급을 2단계로 줄였다면 요구사항의 “부서장 승인”과 “법적 근거 확인”이 한 칸에 들어간다. 그러면 승인만 받고 법적 근거 없이 가구 단위 체납 피처의 값을 여는 요청이 정책상 허용된다 — 등급 병합이 통제를 삭제하는 지점이다. 반대로 등급을 여섯 단계로 늘렸다면, 인접한 두 등급의 통제와 근거 조문이 실제로 다른지 다시 확인한다.

표 13.11의 실패율 상한이 실측 3일분에 어떻게 적용되는가. 표 13.9의 v1 실패율 (0.05-0.0588-0.0455)에 자기 상한을 적용하면 다음과 같이 판정된다.

내가 정한 상한	걸리는 날(v1 기준 3일 중)
6%	0일
5%	1일 — 2026-07-02(0.0588)
4.5%	3일 모두(0.05-0.0588-0.0455)

1.5%포인트 차이가 “게이트가 한 번도 작동하지 않음”과 “매일 작동함”을 결정한다. 예측 칸에 “1일”을 적고 근거로 “5%가 관례라서”를 적었다면, 그것은 값을 고른 것이 아니라 관례를 옮겨 적은 것이다. 상한을 다시 쓰되 이번에는 인력 배치가 견딜 수 있는 오차에서 역산한다.

1단계 표 13.10-13.11로 돌아가 “이 설계가 깨지는 조건” 칸을 다시 쓴다. 처음에 비워 두었거나 막연히 적었다면, 지금은 실측 판정과 실패율 분포를 근거로 작성할 수 있다. 제출하는 설계표는 이 수정을 거친 최종본이다.

관찰

카탈로그·감사표의 모든 수치가 8.7장 실측에서 유도된다 — 이 실습은 새 데이터를 만들지 않고, 8장이 남긴 정의와 7장이 확정한 집계에 거버넌스 계층을 씌웠을 뿐이다.

거부 4건이 각기 다른 통제 지점을 드러낸다 — 하나라도 빠지면(예: 미등록 역할을 기본 허용으로 두면) 통제에 구멍이 난다. 기본값은 거부(deny by default)여야 한다.

정의 변경이 하루만 바꾼 것은 우연이 아니라 7장의 중복 제거 실측의 필연이다 —

dup_removed가 0이 아닌 날에만 valid≠physical이므로. 실측이 손 검산으로 재현되는 설계가 8.9장부터 이어진다.

감사 점검표를 수작업으로 쓰지 않고 로그에서 생성한 것이 요점이다 — 문서와 실물의 불일치(8장 정의서와 같은 철학)를 “문서를 실물에서 생성”으로 막는다.

이 장은 8장부터 이어진 “침묵 실패” 목록에 두 항목을 추가한다 — 카탈로그 밖에서 별도로 사용되는 피쳐(shadow feature)와 기록되지 않는 조회다. 유실(4장)·폐기(6장)·정제 drop(7장)·빈 적재와 행 탈락(8장)이 “비어 있는데 알리지 않는” 침묵이었다면, 거버넌스 없는 접근은 “일어났는데 남지 않는” 침묵이다. 카탈로그 게이트와 접근 로그는 이 침묵을 가시화한다 — 등록되지 않은 피쳐는 거부로, 모든 조회는 기록으로 드러난다.

실습과 운영의 간극(축소 지점 목록)

카탈로그 도구(SQLite ↔ DataHub·OpenLineage), 인증(역할 문자열 가정 ↔ 실제 인증·인가 연동), 피쳐 규모(2종 ↔ 수백 종·다중 서비스), 계보 추적(서비스 단위 소비자 ↔ 필드 단위 lineage). 축소된 것은 규모와 인프라이지 규율이 아니다 — 카탈로그·접근 로그·품질 게이트·변경 이력은 어느 규모에서든 같은 형태로 적용된다.

4단계 위험 식별 — 설계와 증거를 감사 언어로 옮기기

앞의 세 단계에서 설계 근거(1단계)와 실행 증거(2·3단계)를 모두 확보했다. 감사가 요구하는 것은 그 둘을 이어 붙인 형태다 — “무엇이 잘못될 수 있는가, 그렇게 판단한 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가”. 아래 표의 앞 네 행은 실행 증거에서, 뒤 두 행은 1단계 설계에서 나온다.

위험	실습에서의 근거	대응 방향
shadow feature(카탈로그 밖 피쳐)	미등록 피쳐 요청이 거부됨(표 13.4)	카탈로그 등록을 접근의 전제로 강제
목적 외 값 조회	감사 역할 값 조회 거부	역할×행위 단위 권한 + 목적 제한
감사 대응 지연	점검표를 로그에서 자동 생성	조회와 단일 로그 지점 + 자동 점검
정의 변경의 은밀한 스큐	v1→v2가 하루 값 변경, 영향 소비자 식별	버전 발행 + 소비자 통보 + 승인 기록
등급 병합으로 통제가 사라짐	3단계 대조: 등급을 2단계로 줄이면 부서장 승인과 법적 근거 확인이 한 칸에 들어감	적용 근거 조문이 달라지는 지점을 기준으로 등급 경계를 다시 설정(표 13.10 재작성)
임계값 오설정으로 게이트가 무력화	3단계 대조: v1 실패율 3일치에 상한 6%면 0일, 4.5%면 3일 전부 걸림	상한을 인력 배치가 견딜 수 있는 오차에서 역산하고 소유자 승인으로 확정

셋째 열이 비어 있거나 “관리한다” 같은 말로 채워지면 그 행은 감사 문서에서 유용하지 않다. 대응 방향에는 **바꿀 정책값이나 바꿀 설계 결정**이 들어가야 한다.

제출물 3종: ① 표 13.10·13.11 설계표(3단계 수정을 반영한 최종본), ② `ch12_governance_report.json`, ③ 위 위험 식별 표. ①이 설계, ②가 증거, ③이 둘을 이은 감사 자산이다.

보충(전문가 트랙 포인터)

민감 피처 승인 워크플로: 개인정보 보호법 제23조(민감정보 처리 제한)에 해당하는 피처(건강·성생활 등)는 별도 동의·법령 근거·목적 제한·보존 기한이 붙는다. 접근 정책에 “소유자 승인 단계”를 추가하고, 승인 없는 값 조회를 거부하며, 승인·조회를 모두 로그로 남기는 것이 기술적 집행 지점이다. 실습의 등급 설계(표 13.5)가 그 골격이며, 승인 워크플로는 대체로 네 단계로 설계한다.

신청: 소비자가 목적·법적 근거와 함께 접근을 신청한다.

심사: 소유자가 목적 적합성을 심사해 승인·반려한다.

부여: 승인 시 만료 기한이 붙은 조회 권한을 부여한다.

회수: 만료·목적 종료 시 권한을 회수하고 보존 기한 경과 후 삭제한다.

이 네 단계가 전부 로그로 남아야 감사에서 “누가 왜 이 민감 피처를 볼 수 있었는가”에 답할 수 있다. 실습의 접근 판정 로직에 “소유자 승인 여부” 조건을 한 줄 더하는 것이 2·3등급 피처로의 자연스러운 확장이다.

lineage 추적 도구 비교: 카탈로그가 “무엇인가”라면 계보(lineage)는 “어디서 와서 어디로 가는가”다. OpenLineage(개방 표준), DataHub(메타데이터 카탈로그+계보) 등이 원천 테이블부터 피처·모델까지의 변환 그래프를 기록한다 — 실습의 서비스 단위 소비자 추적을 필드·컬럼 단위로 정밀화한다. 7장 롤업의 계층화(하위 확정 산출물만 읽기)가 계보의 한 단면이다.

피처 폐기와 모델 영향 분석: 소비자가 없는 피처만 안전하게 폐기한다. 카탈로그의 소비자 그래프가 비어 있으면 폐기, 비어 있지 않으면 폐기 전 이전 계획을 세운다 — 변경 관리(13.4)의 극단 사례가 폐기다.

핵심 용어

피처 거버넌스: 재사용되는 피처를 카탈로그·접근 이력·품질 변경으로 관리하는 규율

피처 카탈로그: 정의·소유자·원천·개인정보·소비자·버전을 모은 피처 등록소

소비자(consumer): 피처를 읽는 모델·서비스 — 변경 영향 분석의 근거

RBAC: 역할 기반 접근 제어 — 역할×행위 단위로 권한을 배정

목적 제한: 등록된 목적 밖의 이용을 막는 통제(개인정보 보호법 제18조)

사용 이력(access log): 허용·거부 무관하게 남기는 조회 기록 — 감사의 근거

기본 거부(deny by default): 정책에 없는 역할·피처는 거부가 기본값

품질 규칙(게이트): 계약 위반 값을 배포 전에 차단(완전성·정확성·일관성)

변경 관리: 정의 변경을 버전으로 내고 영향 소비자를 식별·통보·승인

CACE: 하나를 바꾸면 전부 바뀐다 — 피처 변경의 파급(Sculley et al., 2015)

계보(lineage): 원천부터 피처·모델까지의 변환 그래프(보충)

품질 게이트 vs 드리프트 감시: 전자는 계약 위반 차단, 후자는 분포 이동 감시(11장) — 다른 층

검수 질문 한 줄 요약

발주·검수 문서에서 이 장의 내용은 네 질문으로 압축된다: ① 재사용 피처가 카탈로그에 등록되어 소유자·개인정보·소비자를 갖는가, ② 피처 조회가 목적 제한과 함께 로그로 남고 그 로그에서 감사 점검표가 자동으로 나오는가, ③ 품질 규칙(게이트)이 배포 전에 계약 위반을 막는가, ④ 정의 변경에 버전·영향 소비자 식별·승인 기록이 있는가. 넷 다 답이 없으면 “피처를 관리한다”는 문장은 구호일 뿐이다.

다음 장 예고

이 장까지 부품이 다 모였다 — 수집·품질(1~2장), 아키텍처(3장), 신뢰성(4~5장), 스트림·배치(6~7장), 피처(8장)와 그 거버넌스(13장), 모델의 등록·배포·감시(9~11장), 부하(12장). 14장은 이 부품들을 하나의 운영 흐름으로 재조립한다. 14장의 통합 MVP는 피처가 1종·소비자가 1개라 8·13장의 장치를 축소했지만 (표 14.5), 그 축소가 성립하는 조건을 아는 것이 축소를 쓸 자격이었다 — 이 장은 그 조건에서 벗어나는 순간(피처 다중·소비자 다수)에 무엇이 필요한지를 채운다.

참고문헌

- Feast. (n.d.). *Feast Documentation*. <https://docs.feast.dev/>
- DataHub. (n.d.). *DataHub Documentation*. <https://datahubproject.io/docs/>
- OpenLineage. (n.d.). *OpenLineage*. <https://openlineage.io/>
- Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data. <https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS. <https://proceedings.neurips.cc/paper/2015/hash/86df7dcfd896fc2674f757a2463eba-Abstract.html>
- 법제처. *개인정보 보호법* (제18조 목적 외 이용·제공 제한, 제23조 민감정보 처리 제한). <https://www.law.go.kr/법령/개인정보보호법>
- 행정안전부·한국지능정보사회진흥원. *공공데이터 품질관리* (3대 품질지표: 정확성·완전성·일관성). <https://www.data.go.kr/ugs/selectPublicDataQlityView.do>
- DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge* (데이터 거버넌스·메타데이터 관리·데이터 품질 지식 영역 — 배경 원칙). <https://www.dama.org/cpages/body-of-knowledge>
- NIST. (2023). *AI Risk Management Framework (AI RMF 1.0)* (GOVERN 기능: 책임·기록·정책 — 방법론 차용, 1장 인용 연장). <https://www.nist.gov/itl/ai-risk-management-framework>
- 개인정보보호위원회. *개인정보의 안전성 확보조치 기준* (고시 — 제5조 접근 권한의 관리, 제8조 접속기록의 보관 및 점검. 개인정보 처리에 대한 배경 원칙). <https://www.law.go.kr/행정규칙/개인정보의안전성확보조치기준>
- Data Contract Specification. *Data Contract Specification* (스키마·의미·품질·변경 정책의 기계가독 계약 — 방법론 차용, 1장 데이터 계약 개념의 현행 형식화). <https://datacontract.com/>